P-Dot: Cross-Team Handoff Specifications & Future Roadmap

> **Author**: Person C (AI Layer)
> **Audience**: Person A (Frontend), Person B (Backend / Database / Infra), Product Stakeholders
> **Project Phase**: MVP Complete $\rightarrow$ Production Integration & Future Evolution
> **Date**: October 2026

Table of Contents

1. Executive Handoff Context
2. Specifications for Person A (Frontend & User Experience)
3. Specifications for Person B (Backend, Database & Cloudflare Infra)
4. Future System Architecture Upgrades (Post-Prototype Evolution)
    - 4.1 Planner–Worker–Evaluator Architecture (Dynamic Tool Hopping)
    - 4.2 Voice Interface & Conversational Duplex Mode
    - 4.3 Expansive Tool Ecosystem (Calendar, Email, Local Notes, Sandboxes)
    - 4.4 Proactive Agentic Routines & Morning Briefings
5. Summary Handoff Matrix

---

1. Executive Handoff Context

The AI Layer for P-Dot has completed its foundational milestones:

- **Inference & Tiering**: Nebius Token Factory with primary model `nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B` and transparent fallback to `Llama-3.3-70B-Instruct`.
- **Autonomous Multi-Hop Tooling**: Search, document retrieval, and memory persistence with Chain-of-Thought extraction.
- **Vector RAG**: 4096-dimensional embeddings with `Qwen/Qwen3-Embedding-8B` over personal documents.
- **Cross-Session Memory**: Vector-backed long-term memory with hybrid pre-retrieval.
- **Hallucination Prevention**: Explicit grounding invariants and a 50-prompt automated evaluation benchmark maintaining a 100% pass rate.

To deliver a polished end-to-end user experience and scale into a true personal assistant, this document defines **exact requirements from Frontend (Person A)**, **Infrastructure/Database needs from Backend (Person B)**, and a **strategic blueprint for future capabilities**.

---

2. Specifications for Person A (Frontend & User Experience)

The AI Layer communicates via **NDJSON (Newline-Delimited JSON) streams** over `POST /api/chat`. Person A should implement frontend components matching the streamed event protocol.

2.1 NDJSON Stream Event Contract

Event Type	Payload Schema	Frontend UI Behavior
`thought`	`{ type: "thought", content: string }`	Stream into an expandable **"Thinking Process"** accordion tray above the response.
`delta`	`{ type: "delta", text: string }`	Progressively append to the active message Markdown view.
`search`	`{ type: "search", status: "used" | "failed", sources: Array<{ title, url }> }`	Display a **"Searching the web..."** badge and render source links.
`action`	`{ type: "action", action: { type, title, ... } }`	Render an interactive **Action Card** with *Approve*, *Edit*, and *Cancel* buttons.
`done`	`{ type: "done", conversationId, messageId }`	Mark streaming complete, enable user input, and refresh conversation sidebar title.
`error`	`{ type: "error", code, message }`	Render an inline retryable toast or error banner.

2.2 Immediate Frontend UI Deliverables

1. FE-02: Rich Markdown & Interactive Citation Badges

- **Current State**: Messages are displayed as plain text in `MessageView.tsx`.
- **Requirement**:
  - Integrate a safe Markdown parser (e.g. `react-markdown` with `remark-gfm` and `rehype-sanitize`).
  - Implement syntax highlighting for code blocks with a one-click **Copy Code** button.
  - Parse citation references like `[1]`, `[2]` and render them as **clickable superscript badges** that automatically scroll to and highlight the corresponding source card in the source list.

2. FE-03: Expandable Thought / Reasoning Drawer

- **Requirement**:
  - Render live Chain-of-Thought reasoning from `{ type: "thought" }` events in an expandable, subtle accordion tray (*"Show reasoning (3 steps)..."*).
  - Keep it collapsed by default once generation completes so the primary answer remains front and center.

3. FE-04: Interactive Action Cards (`user_tasks`, `user_reminders`)

- **Requirement**:
  - When the AI Layer emits `{ type: "action" }`, render an interactive card:
    - **Header**: Action type (e.g. *"Set Reminder"*, *"Create Task"*).
    - **Body**: Date, time, description, or parameters.

        - **Action Buttons**: **Approve** (calls API to commit to DB), **Edit** (opens modal to adjust values), and **Dismiss / Cancel**.
    - Provide immediate visual confirmation upon approval (*"Reminder saved for 9:00 AM"*).

4. FE-05: "What P-Dot Remembers" (Memory Management UI)

- **Requirement**:
    - A dedicated view in Settings or a sidebar panel that calls `GET /api/memories`.
    - Group memories by category (`preference`, `work`, `personal`, `fact`).
    - Provide a quick delete button (`DELETE /api/memories/[id]`) and inline text editing (`PATCH /api/memories/[id]`), giving users complete transparency and privacy control.

5. FE-06: Document Ingestion Screen (Personal RAG)

- **Requirement**:
    - A drag-and-drop document upload interface supporting `.txt`, `.md`, and `.pdf` files.
    - Sends files to `POST /api/documents` (which triggers chunking and embedding).
    - Displays indexing status (*"Uploading"*, *"Embedding 14 chunks..."*, *"Ready"*).

6. FE-07: Continuation & Stop Controls

- **Requirement**:
    - When generation is running, replace the Submit button with a **Stop Generation** button that aborts the `AbortController` signal.
    - If a response concludes with a continuation prompt, provide a quick **"Continue"** chip that automatically sends *"continue"* into the chat.

---

3. Specifications for Person B (Backend, Database & Cloudflare Infra)

The AI Layer depends on PostgreSQL (`pgvector`), Supabase Auth, and Cloudflare Worker runtime stability.

3.1 Database Migrations & Vector Policies

1. **Apply Migration `20261020000002_conversation_update_and_vector_search_path.sql` in Production**:
    - Ensures `match_user_memories` and `match_document_chunks` RPC functions execute with `set search_path = public, pg_temp` so the `<=>` cosine distance operator functions correctly.
    - Grants column-level update permissions on `conversations(title, updated_at)` to `authenticated` users, allowing background AI titling.

2. **Schema for AI-07 Action Cards (`user_tasks`, `user_reminders`)**:
    - Provide database tables for tasks and reminders:

```
create table if not exists public.user_reminders (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null default auth.uid() references auth.users (id) on delete cascade,
  title text not null,
  due_at timestamptz not null,
  status text not null default 'pending' check (status in ('pending', 'completed', 'dismiss
  created_at timestamptz not null default now()
);
alter table public.user_reminders enable row level security;
create policy user_reminders_own on public.user_reminders
  for all to authenticated using ((select auth.uid()) = user_id);
```

3.2 Supabase Storage Bucket Configuration

- **Bucket Name**: `documents` (private).
- **Storage RLS**: Users can only upload, read, and delete objects stored within their own user folder:

```
create policy "User Document Storage Isolation" on storage.objects
  for all to authenticated
  using (bucket_id = 'documents' and (storage.foldername(name))[1] = auth.uid()::text);
```

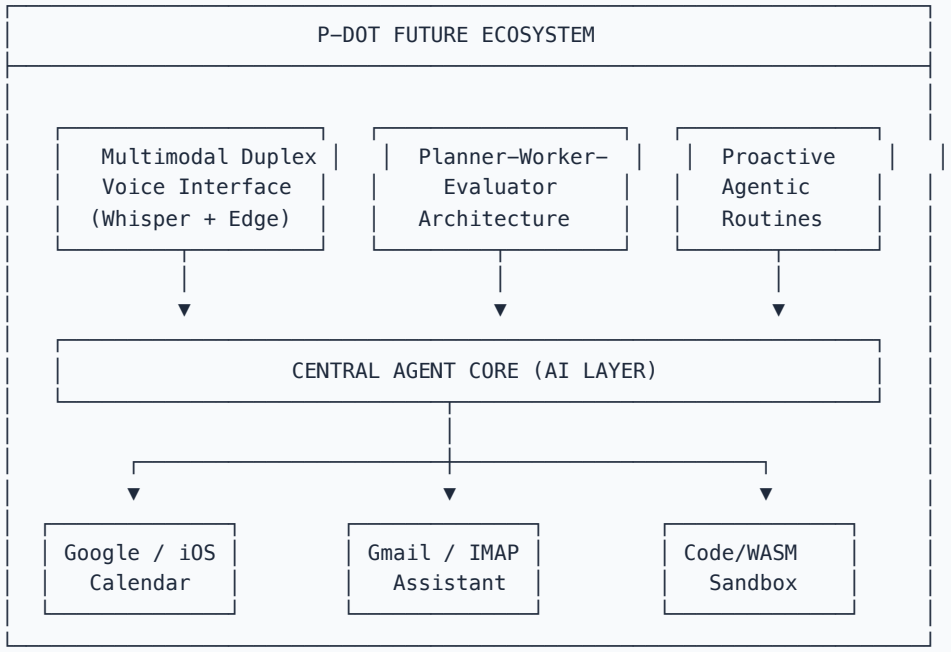
3.3 Cloudflare Workers & Production Edge Tuning

- **Streaming Keep-Alive Heartbeats**:
  - Cloudflare Workers enforce an edge execution timeout on idle connections.
  - During multi-hop operations (e.g. document embedding + Tavily search taking >5 seconds), ensure the HTTP stream flushes a comment heartbeat (`: ping\n\n`) every 3 seconds to keep edge tunnels alive.
- **Environment Secrets Verification**:
  - Ensure the following variables are populated in the Cloudflare Dashboard (`npx wrangler secret put`):
    - `NEBIUS_API_KEY` : API token.
    - `NEBIUS_MODEL` : `nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B` .
    - `NEBIUS_FALLBACK_MODEL` : `meta-llama/Llama-3.3-70B-Instruct` .
    - `NEBIUS_MAX_OUTPUT_TOKENS` : `4096` .
    - `TAVILY_API_KEY` : Web search key.

---

4. Future System Architecture Upgrades (Post-Prototype Evolution)

Beyond the initial prototype, four major architectural upgrades will elevate P-Dot into a true autonomous personal assistant:



4.1 Planner–Worker–Evaluator Architecture (Dynamic Tool Hopping)

- **Objective**: Remove the static hop limit (`MAX_TOOL_HOPS = 3`) to enable complex, multi-step research while preventing runaway token usage and infinite loops.
- **How It Works**:
    1. **Planner Agent**: Analyzes the user's prompt up-front. Categorizes complexity and allocates an initial dynamic budget (e.g., 0 hops for chit-chat, 5 hops for deep research).
    2. **Worker Agent**: Carries out the individual tool calls (`search_personal_docs`, `search_web`, `save_memory`).
    3. **Evaluator / Critic Agent**: After each tool observation, evaluates: *"Is the user's goal answered?"*
        - If **YES**: Terminates tool calls early and triggers final synthesis.
        - If **NO**: Identifies the missing data point and grants 1 additional hop.
    4. **Circuit Breaker**: Computes a fingerprint hash of `(tool_name, args)`. If a duplicate call is attempted or 2 consecutive calls yield 0 information gain, the breaker trips and forces synthesis.

4.2 Voice Interface & Conversational Duplex Mode

- **Objective**: Transform P-Dot from a keyboard-only chat interface into an omnipresent voice assistant accessible on mobile and desktop.
- **Technical Architecture**:
    1. **Voice Input (STT)**:
        - Edge-accelerated OpenAI Whisper or Web Speech API integration.
        - Voice Activity Detection (VAD) to detect when the user stops speaking naturally.
    2. **Streaming Text-to-Speech (TTS)**:
        - Integration with low-latency streaming TTS (e.g., ElevenLabs Turbo v2 or Cartesia Sonic).

    - Streams audio chunks simultaneously as text deltas are generated, achieving sub-second speech response.

3. **Duplex Interruption**:
    - Client-side microphone stream automatically interrupts assistant audio playback if the user begins speaking.

4.3 Expansive Tool Ecosystem (External Data & Productivity)

To become a true personal assistant, P-Dot must interact with the user's external productivity suite:

New Tool	Capabilities	Integration Protocol
Calendar Sync	Query daily agenda, detect meeting conflicts, insert events with reminders.	Google Calendar API / Apple CalDAV OAuth.
Email Assistant	Search email threads, draft replies, summarize newsletter digests.	Gmail REST API / IMAP OAuth with user consent.
Local Notes & Drive	Ingest and index entire folders from Google Drive, Notion, or Obsidian.	Background sync worker with differential vector indexing.
Sandboxed Code Execution	Run Python / JavaScript code snippets to generate charts, calculate financials, and parse CSVs.	Sandboxed WebAssembly (Wasm) or Cloudflare Browser Rendering/Code Engine.

4.4 Proactive Agentic Routines & Morning Briefings

- **Objective**: Shift P-Dot from a **reactive** bot (waits for prompt) to a **proactive** assistant (works in the background).
- **Key Features**:
    1. **Daily Morning Briefing**:
        - At a user-configured time (e.g. 7:30 AM), a scheduled cron job synthesizes:
            - Today's calendar schedule.
            - Pending action items from previous chats.
            - Weather and selected news updates.
        - Delivered via push notification or a prominent dashboard banner.
    2. **Autonomous Memory Consolidation**:
        - Nightly background worker analyzes the past day's conversations, resolves conflicting preferences, and consolidates fragmented memories into unified user profile facts.

---

5. Summary Handoff Matrix

Team Role	Primary Responsibility	Next Milestone
Person A (Frontend)	Render Markdown citations `[n]`, thought accordions, action cards, and memory/doc screens.	**FE-02 & FE-04**
Person B (Backend/Infra)	Verify production Supabase migrations, storage buckets, and Worker streaming keep-alives.	**Database & Storage RLS**
Person C (AI Layer)	Calendar/action proposal execution, Planner-Worker-Evaluator loop, and voice agent integration.	**AI-07 & Multi-Agent Loop**